

Zackery Spears

June 24, 2026

Assignment Name: CS 470 Final Reflection

[https://www.youtube.com/watch?v=OQ2999h5fns&list=PL-](https://www.youtube.com/watch?v=OQ2999h5fns&list=PL-E1cJGJUzb9wvNk9Bd5g32egr2Qa7i6h)

[E1cJGJUzb9wvNk9Bd5g32egr2Qa7i6h](https://www.youtube.com/watch?v=OQ2999h5fns&list=PL-E1cJGJUzb9wvNk9Bd5g32egr2Qa7i6h)

CS 470 Final Reflection

Experiences and Strengths

CS 470 has helped me move closer to my professional goal of becoming a software developer by giving me hands-on experience with cloud-based application development. Before this course, I understood full stack development mostly from the perspective of building an application and running it locally. This course helped me understand how an application can be prepared for real-world deployment by using containers, cloud storage, serverless functions, databases, APIs, and security permissions. That experience is important because many software development jobs now expect developers to understand not only how to write code, but also how that code is deployed, scaled, protected, and maintained in a cloud environment.

One of the most valuable skills I developed in this course was containerization. Working with Docker and Docker Compose helped me understand how a MEAN stack application can be packaged so that the front end, back end, and database can run in consistent environments. This is useful in professional development because containers reduce the common problem of an application working on one computer but failing on another. I also learned how containerization supports collaboration because developers can share a repeatable environment instead of relying only on local machine setup.

I also gained experience with AWS cloud services. I hosted a front-end application using Amazon S3 static website hosting, created serverless functions with AWS Lambda, connected those functions to API Gateway, and used DynamoDB as a cloud database. In the final stages of the project, I tested CRUD operations through Lambda functions and API Gateway methods for resources such as Questions and Answers. This helped me better understand how cloud applications are organized into services that communicate through APIs instead of depending on one large, tightly coupled application.

Another important skill I developed was troubleshooting. During the course, I had to work through issues involving table names, Lambda runtime syntax, API Gateway stages, CORS, IAM permissions, and database access. These problems were frustrating at times, but they helped me build confidence in reading error messages, checking configurations, testing one component at a time, and documenting evidence of progress. That problem-solving process is one of my strengths as a developer. I am willing to keep testing, researching, and adjusting my work until the application functions correctly.

My strengths as a software developer include persistence, attention to detail, and a growing ability to connect different parts of a system together. I am still developing as a programmer, but this course showed me that I can work through a complete cloud migration process and understand the purpose of each major component. I am strongest when I can break a problem into smaller steps, test each step, and build toward a complete solution. My experience in this course prepares me for entry-level roles such as junior software developer, full stack developer, cloud application developer, application support developer, or DevOps/cloud support associate. These roles require a combination of programming ability, troubleshooting,

documentation, and understanding of deployment environments, which are skills I practiced throughout CS 470.

Planning for Growth

Planning for future growth requires thinking about more than whether the application works today. A cloud-based application must be able to handle more users, more data, more API requests, and more failures without becoming unreliable or too expensive. Based on what I learned in CS 470, I would plan future growth by separating the application into smaller services, using cloud monitoring tools, designing for error handling, and choosing services that support elasticity and pay-for-service pricing.

Microservices and serverless architecture can create management and scaling efficiencies because each part of the application can be separated by responsibility. For example, the application could use one service for questions, another service for answers, another service for user authentication, and another service for reporting or analytics. Each service could be updated, tested, and scaled independently. This would make the application easier to maintain because a change to one service would not require redeploying the entire application. Serverless functions such as AWS Lambda are useful for this approach because each function can focus on a specific task, such as getting one record, scanning a table, inserting a question, updating an answer, or deleting a record.

To handle scale, I would use services that can automatically respond to demand. The front end could continue to use static hosting through S3, and a production version could add a content delivery network to reduce latency for users in different locations. API Gateway could manage incoming API requests and route them to Lambda functions. Lambda would automatically scale by running more function instances as requests increase. DynamoDB could

be configured with on-demand capacity or auto scaling so the database can handle changes in traffic without requiring manual server management. This design would allow the application to support spikes in usage more easily than a single server running the entire application.

Error handling would need to be planned at every layer. API Gateway should return clear status codes so the client application can understand whether a request succeeded or failed.

Lambda functions should validate input, handle missing records, catch exceptions, and log useful error messages. For operations that might fail temporarily, the application could use retries with limits so it does not create an endless loop of failed requests. For more advanced growth, failed events could be sent to a dead-letter queue for later review. CloudWatch logs and metrics would also be important because they would help identify slow functions, repeated failures, high error rates, or unexpected traffic patterns.

Cost prediction would involve estimating usage for each cloud service. For a serverless design, I would estimate the number of API requests, average Lambda execution time, memory allocation, DynamoDB reads and writes, S3 storage, S3 requests, and data transfer. I would also use AWS pricing tools, budgets, alarms, and usage metrics to compare the estimated cost with the actual cost after deployment. Load testing would help predict costs because it would show how the application behaves under higher traffic. Monitoring real traffic over time would make the cost estimate more accurate.

Containers and serverless have different levels of cost predictability. Containers can be more predictable when an application has steady traffic because the team can estimate the cost of running a fixed number of containers or servers for a month. However, containers may still cost money even when traffic is low because resources are running continuously. Serverless can be less predictable because cost depends on the number of requests, execution time, and resource

usage. At the same time, serverless can be more cost efficient for applications with variable or unpredictable traffic because the company pays mainly when the service is used. For my application, serverless would be a strong option during early growth because it reduces the need to manage infrastructure and supports automatic scaling.

There are several pros and cons that would affect expansion plans. The main advantages of serverless are automatic scaling, reduced infrastructure management, faster deployment of small services, and pay-for-use pricing. The disadvantages include cold starts, service limits, vendor lock-in, and the need for strong monitoring because the system is distributed across multiple services. Containers provide more control over the runtime environment, can be more portable across platforms, and may be easier to manage for applications with consistent workloads. However, containers often require more responsibility for orchestration, patching, scaling rules, and infrastructure management. These tradeoffs would determine whether the application should remain mostly serverless, move toward containers, or use a hybrid approach.

Elasticity and pay-for-service pricing play a major role in decision making because they allow the application to grow without requiring the company to purchase and maintain extra capacity before it is needed. Elasticity means the application can expand during high demand and shrink during low demand. Pay-for-service pricing means the company can align cost with actual usage. These concepts are especially valuable for a growing application because user demand may not be predictable. Instead of overbuilding infrastructure, the team can start with scalable managed services, watch real usage, and adjust the architecture as the application becomes more popular.

Conclusion

Overall, CS 470 helped me understand how full stack development connects with cloud deployment and long-term application planning. I learned how containers, APIs, serverless functions, cloud databases, and cloud permissions work together to support a modern application. I also learned that cloud development requires careful thinking about security, cost, scalability, error handling, and maintainability. These skills make me a stronger and more marketable software developer because I can discuss not only how to build an application, but also how to prepare it for real users and future growth.